

Time and space complexity analysis

4.

- The time complexity is $O(n^2)$ where n is the number of nodes in the given adjacency matrix because everytime I run the algorithm I am processing one node but for each node I am also processing all of its neighbors which can be up to n .
- The space complexity is $O(n^2)$ as well where n is the number of nodes in the adjacency matrix because for each node I have my separate lists created and updated in loops such as visited, parents, my queue, and the path I am returning all of which will be up to n in them since representing the nodes given in the graph. I am given a matrix which is n^2 since its a double array but I create nothing with it so everytime I run the algorithm I am creating at most $O(n)$ space

6.

- The time complexity is $O(V+E)$, classic, where V is the number of courses you have to take and E is the number of prerequisites because everytime I run I visit each node once and then also each edge everytime I do the recursion. So the V is im visiting each node once and the E is me visiting the edges to get to each of the nodes V .
- Space complexity is also $O(V+E)$ here V is the number of courses you have to take and E is the number of prerequisites because I make that list of list and integer the `List<List<Integer>>` which I store all the connection(edges) in and also make the visited array which stores all the nodes in there and same with the stack but both are both the same V for amount of nodes given so I can chop off constant so its still my $V+E$